Atención y la Arquitectura Transformer

De la atención de Bahdanau a los modelos de lenguaje masivos

Sergio Hernández, PhD
shernandez@ucm.cl



Magíster en Data Science — Universidad Católica del Maule

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

¿Dónde Estamos?

Clase 3.2

- Embeddings vectoriales.
- RNN, LSTM y GRU: procesan la secuencia **paso a paso**.
- Arquitectura codificador–decodificador (seq2seq).

Dos problemas de las RNN

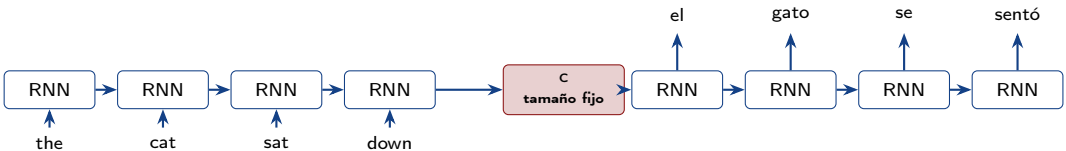
1. **Cuello de botella**: toda la fuente se comprime en un único estado de tamaño fijo.
2. **Secuencialidad**: el paso t espera al paso $t-1$; no se paraleliza en GPU.

Esta clase

1. Consultas, claves y valores.
2. Atención por similitud (Nadaraya–Watson).
3. Funciones de puntaje y *masked softmax*.
4. Atención de Bahdanau.
5. Auto-atención: simple, con pesos entrenables y causal.
6. Atención multi-cabeza.
7. Codificación posicional.
8. El Transformer.
9. Vision Transformer.
10. Preentrenamiento: BERT, T5, GPT.
11. Laboratorio 3.3.

Contenido basado en *Dive into Deep Learning*, cap. 11 (d2l.ai).

El Cuello de Botella del Modelo Seq2Seq



- Toda la oración fuente debe caber en $c \in \mathbb{R}^h$, sin importar su largo.
- El rendimiento cae al aumentar la longitud de la entrada.
- **Idea** (Bahdanau, Cho y Bengio, 2014): en cada paso del decodificador, *mirar* los estados del codificador que sean relevantes en lugar de depender de un único vector.

Consultas, Claves y Valores

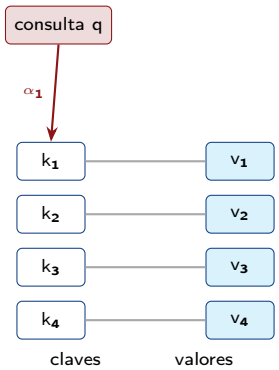
Una **base de datos** de m pares clave–valor:

$$\mathcal{D} = \{(\mathsf{k}_1, \mathsf{v}_1), \ldots, (\mathsf{k}_m, \mathsf{v}_m)\}$$

y una **consulta** q.

$$\text{Attention}(\mathsf{q}, \mathcal{D}) = \sum_{i=1}^{m} \alpha(\mathsf{q}, \mathsf{k}_i)\, \mathsf{v}_i$$

- $\alpha(\mathsf{q}, \mathsf{k}_i) \in \mathbb{R}$: **pesos de atención**.
- La operación se llama *attention pooling*.
- Es una selección **diferenciable** sobre un conjunto de tamaño variable.



Normalización y Casos Especiales

Para que los pesos formen una combinación convexa se aplica **softmax** a una función de puntaje $a(\mathsf{q}, \mathsf{k})$:

$$\alpha(\mathsf{q}, \mathsf{k}_i) = \frac{\exp\left(a(\mathsf{q}, \mathsf{k}_i)\right)}{\sum_{j=1}^{m} \exp\left(a(\mathsf{q}, \mathsf{k}_j)\right)} \qquad \alpha_i \geq 0, \quad \sum_i \alpha_i = 1$$

Casos especiales

- Un peso $= 1$ y el resto 0: consulta clásica a una base de datos.
- Todos los pesos $= 1/m$: *average pooling*.
- Pesos no negativos: la salida está en el cono convexo de los valores.

¿Por qué softmax?

- Es diferenciable y sus gradientes no se anulan.
- La atención no diferenciable (selección dura) requiere aprendizaje por refuerzo.
- Todos los mecanismos de atención populares usan softmax.

Atención por Similitud: Regresión de Nadaraya–Watson

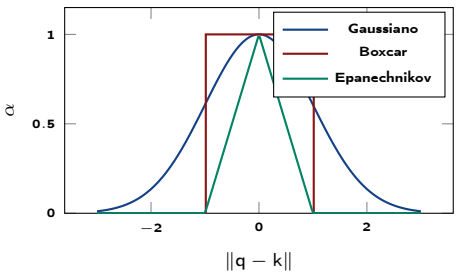
Precursor de la atención (1964): un promedio de las salidas observadas, ponderado por un **núcleo** (kernel).

$$f(x) = \sum_{i=1}^{n} \frac{K(x - x_i)}{\sum_j K(x - x_j)} y_i$$

En lenguaje de atención

- **consulta** = punto donde se predice
- **claves** = entradas observadas x_i
- **valores** = salidas observadas y_i

No requiere entrenamiento. Para clasificación, v_i es el one-hot de y_i.



Nadaraya–Watson: Qué Observar

Experimento de d2l: $y_i = 2\sin(x_i) + x_i + \epsilon$, $\epsilon \sim \mathcal{N}(0, 1)$, 40 puntos en $[0, 5]$.

- Gaussiano, boxcar y Epanechnikov dan ajustes similares.
- Núcleo **constante**: se reduce a la media global de y.
- Ancho menor: curva menos suave, sigue mejor la variación local.
- Un ancho fijo no es óptimo en todas partes (heurística de Silverman para anchos adaptativos).

Limitación

La similitud está **fijada a mano**. En deep learning queremos *aprender* las representaciones de consultas y claves para que la similitud sea útil para la tarea.

Siguiente paso

Reemplazar $\|\mathbf{q} - \mathbf{k}\|$ por una función de puntaje parametrizada.

De Distancia Gaussiana a Producto Punto

Puntaje gaussiano, expandido:

$$a(\mathbf{q}, \mathbf{k}_i) = -\tfrac{1}{2}\|\mathbf{q} - \mathbf{k}_i\|^2 = \mathbf{q}^\top \mathbf{k}_i - \tfrac{1}{2}\|\mathbf{k}_i\|^2 - \tfrac{1}{2}\|\mathbf{q}\|^2$$

- $\frac{1}{2}\|\mathbf{q}\|^2$ es igual para todas las claves: se cancela en el softmax.
- $\frac{1}{2}\|\mathbf{k}_i\|^2$ suele ser casi constante (p. ej. tras normalización en capas anteriores) y se descarta.
- Queda el **producto punto** $\mathbf{q}^\top \mathbf{k}_i$: la similitud mide alineamiento.

Dos opciones populares

1. **Producto punto escalado**: eficiente (una multiplicación de matrices); exige dimensiones iguales de q y k.
2. **Atención aditiva**: permite consultas y claves de dimensiones distintas.

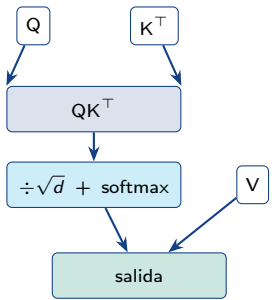
Producto Punto Escalado

$$a(\mathrm{q}, \mathrm{k}_i) = \frac{\mathrm{q}^\top \mathrm{k}_i}{\sqrt{d}} \quad \Longrightarrow \quad \boxed{\mathrm{Attention}(\mathrm{Q}, \mathrm{K}, \mathrm{V}) = \mathrm{softmax}\left(\frac{\mathrm{QK}^\top}{\sqrt{d}}\right)\mathrm{V}}$$

con $\mathrm{Q} \in \mathbb{R}^{n \times d}$, $\mathrm{K} \in \mathbb{R}^{m \times d}$, $\mathrm{V} \in \mathbb{R}^{m \times v}$ y salida en $\mathbb{R}^{n \times v}$.

¿Por qué dividir por $\sqrt{d}$?

Si las componentes de q y k son independientes con media 0 y varianza 1, entonces $\mathrm{q}^\top \mathrm{k}$ tiene varianza d. Dividir por $\sqrt{d}$ la devuelve a 1. Sin esto, para d grande el softmax se satura (un peso ≈ 1) y los gradientes casi se anulan.

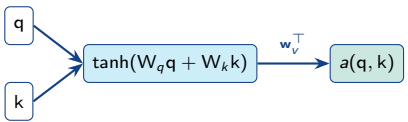


Atención Aditiva

Cuando $\mathrm{q} \in \mathbb{R}^q$ y $\mathrm{k} \in \mathbb{R}^k$ tienen dimensiones distintas:

$$a(\mathrm{q}, \mathrm{k}) = \mathrm{w}_v^\top \tanh\left(\mathrm{W}_q \mathrm{q} + \mathrm{W}_k \mathrm{k}\right) \in \mathbb{R}$$

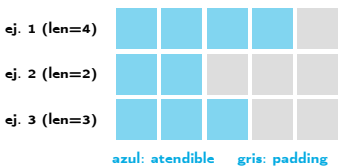
- $\mathrm{W}_q \in \mathbb{R}^{h \times q}$, $\mathrm{W}_k \in \mathbb{R}^{h \times k}$, $\mathrm{w}_v \in \mathbb{R}^h$: parámetros aprendibles.
- Equivale a un MLP de una capa oculta (tanh) sobre la concatenación de q y k.
- Es el puntaje usado originalmente por Bahdanau.



Masked Softmax: Secuencias de Largo Variable

En un minibatch las secuencias se rellenan (*padding*); esas posiciones no deben recibir atención.

- Se reciben las longitudes válidas por ejemplo (1-D) o por consulta (2-D).
- Las posiciones inválidas reciben un puntaje muy negativo ($\approx -10^6$) antes del softmax $\Rightarrow$ peso $= 0$.
- Después del softmax se aplica *dropout* y luego se multiplica por V.



Atención de Producto Punto en PyTorch

```
def masked_softmax(scores, valid_lens):          # scores: (B, Q, K)
    mask = torch.arange(scores.shape[-1])[None, None, :] \
           < valid_lens[:, None, None]
    return torch.softmax(scores.masked_fill(~mask, -1e6), dim=-1)

def attention(Q, K, V, valid_lens=None):
    d = Q.shape[-1]
    scores = Q @ K.transpose(1, 2) / math.sqrt(d)  # (B, n, m)
    weights = masked_softmax(scores, valid_lens)
    return weights @ V, weights                    # (B, n, v)
```

- `@` sobre tensores 3-D es una multiplicación de matrices por lote (*bmm*).
- Todo es diferenciable: se entrena con retropropagación.

Atención de Bahdanau: Contexto Dinámico

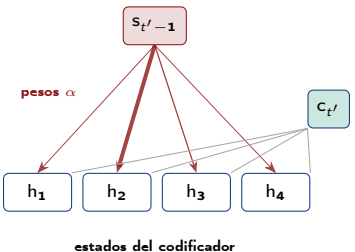
El contexto $c_{t'}$ se recalcula en **cada** paso t' del decodificador.

$$c_{t'} = \sum_{t=1}^{T} \alpha(s_{t'-1}, h_t)\, h_t, \qquad \alpha(s, h_t) = \mathrm{softmax}_t\Big(w_v^\top \tanh(W_q s + W_k h_t)\Big)$$

Roles

- **consulta**: estado previo del decodificador $s_{t'-1}$
- **claves y valores**: estados del codificador $h_1, \ldots, h_T$

Entrada del decodificador: $[\mathrm{Embed}(y_{t'-1}); c_{t'}] \rightarrow$ GRU $\rightarrow$ capa densa $\rightarrow$ logits del vocabulario.



Bahdanau: Arquitectura y Entrenamiento

Componentes

- **Codificador**: RNN que entrega los estados de todos los pasos.
- **Atención**: aditiva, con el estado previo del decodificador como consulta.
- **Decodificador**: embedding $\rightarrow$ atención $\rightarrow$ concatenar $\rightarrow$ GRU $\rightarrow$ densa.
- Las *longitudes válidas* de la fuente enmascaran el padding.

Configuración en d2l (EN$\rightarrow$FR)

- Batch 128, embedding y oculta 256, 2 capas, dropout 0.2.
- lr $= 0.005$, 30 épocas, recorte de gradiente 1.
- Evaluación: BLEU.

Lectura de los pesos

Los mapas de calor muestran qué palabras fuente se miran al generar cada palabra objetivo: **alineamiento** aprendido sin supervisión.

Auto-Atención Simple (sin Pesos Entrenables)

Cada token x_i se reemplaza por un **vector de contexto** z_i: una mezcla de *todos* los tokens de la misma secuencia.

$$\omega_{ij} = x_i^\top x_j \;\Rightarrow\; \alpha_{ij} = \frac{e^{\omega_{ij}}}{\sum_l e^{\omega_{il}}} \;\Rightarrow\; z_i = \sum_{j=1}^{n} \alpha_{ij} x_j \qquad \Longleftrightarrow \qquad Z = \mathrm{softmax}(XX^\top)\,X$$

Ejemplo del laboratorio: *Your journey starts with one step*, con embeddings de 3 dimensiones.
Consulta: `journey`.

	Your	journey	starts	with	one	step
ω_{2j}	0.954	1.495	1.475	0.843	0.707	1.087
α_{2j}	0.139	0.238	0.233	0.124	0.108	0.158

$z_2 = (0.442,\ 0.652,\ 0.568)$

Limitaciones

- No hay nada que aprender: la similitud depende solo de los embeddings de entrada.
- Cada token se atiende mucho a sí mismo ($x \cdot x$ suele ser el mayor producto punto).

Siguiente paso: proyecciones aprendibles.

Auto-Atención con Pesos Entrenables

Tres proyecciones aprendibles dan los roles de **consulta**, **clave** y **valor** (la formulación QKV de esta clase):

$$\mathsf{Q} = \mathsf{X}\mathsf{W}_q, \quad \mathsf{K} = \mathsf{X}\mathsf{W}_k, \quad \mathsf{V} = \mathsf{X}\mathsf{W}_v, \qquad \mathsf{Z} = \mathrm{softmax}\left(\frac{\mathsf{Q}\mathsf{K}^\top}{\sqrt{d_k}}\right)\mathsf{V}$$

- $\mathsf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$, $\mathsf{W}_{q,k,v} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$.
- `SelfAttention_v1`: `nn.Parameter(torch.rand(...))`.
- `SelfAttention_v2`: `nn.Linear`, con sesgo opcional y **mejor inicialización**; es la forma estándar.
- El factor $1/\sqrt{d_k}$ evita que el softmax se sature.

```
def forward(self, x):           # x: (n, d_in)
    keys = self.W_key(x)
    queries = self.W_query(x)
    values = self.W_value(x)

    attn_scores = queries @ keys.T
    attn_weights = torch.softmax(
        attn_scores / keys.shape[-1]**0.5, dim=-1)
    return attn_weights @ values   # (n, d_out)
```

Auto-Atención

Consultas, claves y valores provienen de **la misma** secuencia $x_1, \ldots, x_n$:

$$y_i = f\big(x_i,\ (x_1, x_1), \ldots, (x_n, x_n)\big) \in \mathbb{R}^d$$

Cada token construye su representación *mirando a todos los demás* directamente.

	Complejidad por capa	Operaciones secuenciales	Largo máximo de camino
CNN (k)	$\mathcal{O}(k\, n\, d^2)$	$\mathcal{O}(1)$	$\mathcal{O}(n/k)$
RNN	$\mathcal{O}(n\, d^2)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Auto-atención	$\mathcal{O}(n^2\, d)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$

Ventajas

✔ Paralelismo total en GPU.
✔ Cualquier par de tokens está a un paso: dependencias largas.

Costo

✘ Cuadrático en la longitud n (memoria y tiempo).
✘ **No tiene noción de orden**.

Atención Causal: la Máscara va *Antes* del Softmax

$$\mathrm{Z} = \mathrm{softmax}\left(\frac{\mathrm{QK}^\top}{\sqrt{d_k}} + \mathrm{M}\right)\mathrm{V}, \qquad M_{ij} = \begin{cases} 0 & j \le i \\ -\infty & j > i \end{cases}$$

Ingenuo

softmax $\rightarrow$ multiplicar por la máscara triangular $\rightarrow$ **renormalizar** cada fila.

Eficiente (el que usamos)

Poner $-\infty$ en el futuro y *luego* softmax: como $e^{-\infty} = 0$, las filas ya salen normalizadas, en un solo paso. Ambos dan el mismo resultado.

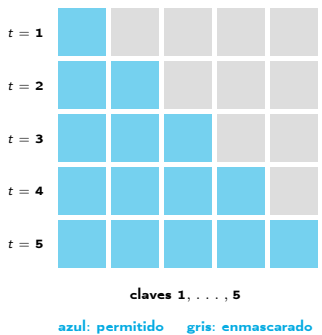
```
mask = torch.triu(torch.ones(T, T), diagonal=1)
scores = scores.masked_fill(mask.bool(), -torch.inf)
weights = torch.softmax(scores / d_k**0.5, dim=-1)
```

`triu(..., diagonal=1)` marca el triángulo *superior estricto*: las posiciones futuras.

Atención Causal: la Máscara Triangular

Para ser **autorregresivo** (decodificador, GPT), el token t solo puede mirar los tokens $\leq t$.

- En entrenamiento se procesa toda la secuencia objetivo en paralelo (*teacher forcing*) con una máscara triangular.
- Equivale a usar longitud válida $= t$ para la consulta t.
- En predicción se generan tokens uno a uno, reutilizando las representaciones ya calculadas (caché).



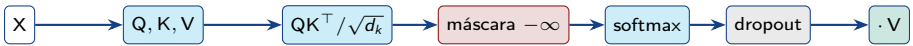
Dropout sobre los Pesos y la Clase `CausalAttention`

Dropout de atención

Después del softmax, durante el **entrenamiento**, cada peso se anula con probabilidad p y los restantes se reescalan por $1/(1-p)$. En evaluación se desactiva.

`CausalAttention` en el lab

- Proyecciones `nn.Linear` W_q, W_k, W_v.
- Dimensión de lote: `queries @ keys.transpose(1, 2)`.
- Máscara en `register_buffer`: viaja a la GPU con el modelo pero no es un parámetro entrenable.



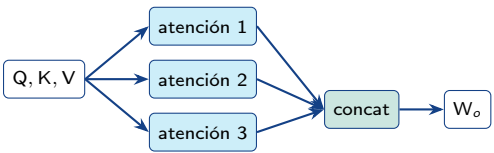
Atención Multi-Cabeza

Varias *cabezas* atienden en paralelo a subespacios aprendidos distintos (relaciones sintácticas, de largo alcance, etc.).

$$\mathrm{h}_i = f\left(\mathrm{W}_i^{(q)}\mathrm{q},\ \mathrm{W}_i^{(k)}\mathrm{k},\ \mathrm{W}_i^{(v)}\mathrm{v}\right) \in \mathbb{R}^{p_v}, \quad i = 1, \ldots, h$$

$$\mathrm{MHA}(\mathrm{q}, \mathrm{k}, \mathrm{v}) = \mathrm{W}_o \begin{bmatrix} \mathrm{h}_1 \\ \vdots \\ \mathrm{h}_h \end{bmatrix} \in \mathbb{R}^{p_o}, \qquad \mathrm{W}_o \in \mathbb{R}^{p_o \times h p_v}$$

- f: atención de producto punto escalado.
- Para controlar el costo: $p_q = p_k = p_v = p_o/h$.
- Así el costo es similar al de una sola cabeza de dimensión completa.
- En el lab: $p_o =$ `d_out`, $p_v =$ `head_dim` $=$ `d_out`$/h$, $\mathrm{W}_o =$ `out_proj`.



Multi-Cabeza: de la Versión Ingenua a la Eficiente

Ingenua: `MultiHeadAttentionWrapper`

h instancias de `CausalAttention` en una `ModuleList`; se recorren con un **bucle** y se concatenan las salidas. La GPU queda subutilizada y la salida mide *h*·`d_out`.

Eficiente: `MultiHeadAttention`

Una proyección grande de tamaño `d_out` y se *divide* en *h* cabezas con `view`: todas se calculan en un único producto matricial por lote. `head_dim = d_out // num_heads`.



Después: $(\text{pesos} \cdot \text{V}) \to (b, h, T, d_h) \xrightarrow{\texttt{transpose}} (b, T, h, d_h) \xrightarrow{\texttt{view}} (b, T, d_{\text{out}}) \to \text{W}_o$.

`MultiHeadAttention` en PyTorch (versión del laboratorio)

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_in, d_out, context_length, dropout, num_heads, qkv_bias=False):
        super().__init__()
        assert d_out % num_heads == 0
        self.d_out, self.num_heads = d_out, num_heads
        self.head_dim = d_out // num_heads
        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)   # idem W_key, W_value
        self.out_proj = nn.Linear(d_out, d_out)                # W_o
        self.dropout = nn.Dropout(dropout)
        self.register_buffer("mask", torch.triu(torch.ones(context_length, context_length), 1))

    def forward(self, x):
        b, T, _ = x.shape
        q = self.W_query(x).view(b, T, self.num_heads, self.head_dim).transpose(1, 2)
        k = ...  # igual con W_key;   v = ...  # igual con W_value   -> (b, heads, T, head_dim)

        scores = q @ k.transpose(2, 3)                         # (b, heads, T, T)
        scores.masked_fill_(self.mask.bool()[:T, :T], -torch.inf)
        w = self.dropout(torch.softmax(scores / k.shape[-1]**0.5, dim=-1))

        z = (w @ v).transpose(1, 2).contiguous().view(b, T, self.d_out)
        return self.out_proj(z)
```

La Auto-Atención No Conoce el Orden

Sin máscara causal, permutar los tokens de entrada solo **permuta la salida**: sea $\mathbf{\Pi}$ una matriz de permutación,

$$\mathrm{softmax}\Big(\frac{(\mathbf{\Pi Q})(\mathbf{\Pi K})^\top}{\sqrt{d}}\Big)\mathbf{\Pi V} = \mathbf{\Pi}\,\mathrm{softmax}\Big(\frac{\mathbf{Q K}^\top}{\sqrt{d}}\Big)\mathbf{\Pi}^\top\mathbf{\Pi V} = \mathbf{\Pi}\,\mathrm{SA}(\mathbf{X})$$

Consecuencia

"el perro mordió al hombre" y "el hombre mordió al perro" producen los mismos vectores (reordenados): el modelo **no distingue** el orden.

Solución

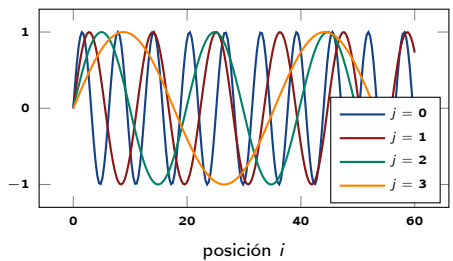
Sumar la codificación posicional: con ella, $\mathrm{SA}(\mathbf{\Pi X} + \mathbf{P}) \neq \mathbf{\Pi}\,\mathrm{SA}(\mathbf{X} + \mathbf{P})$, porque al reordenar cada token recibe la posición de su *nuevo* lugar.

(Ejercicio 3 del laboratorio.)

Codificación Posicional

La auto-atención es invariante al orden $\Rightarrow$ se *suma* una matriz $\mathrm{P} \in \mathbb{R}^{n \times d}$ a la entrada: $\mathrm{X} + \mathrm{P}$.

$$p_{i,2j} = \sin\left(\frac{i}{10000^{2j/d}}\right), \qquad p_{i,2j+1} = \cos\left(\frac{i}{10000^{2j/d}}\right)$$



Ejemplo con $d = 32$: columnas de sin de frecuencia decreciente.

Propiedad de posición relativa

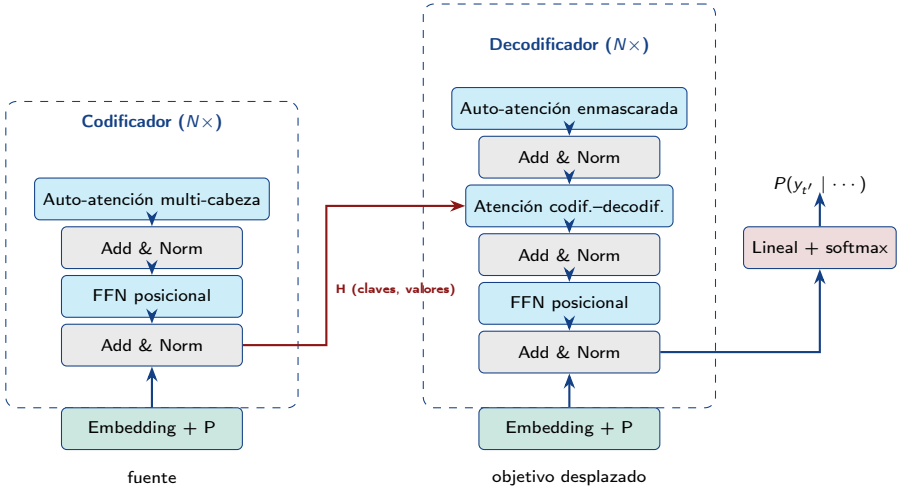
Con $\omega_j = 10000^{-2j/d}$, un giro fijo depende solo del desfase δ:

$$\begin{bmatrix} \cos\delta\omega_j & \sin\delta\omega_j \\ -\sin\delta\omega_j & \cos\delta\omega_j \end{bmatrix} \begin{bmatrix} p_{i,2j} \\ p_{i,2j+1} \end{bmatrix} = \begin{bmatrix} p_{i+\delta,2j} \\ p_{i+\delta,2j+1} \end{bmatrix}$$

El modelo puede aprender a atender por **offset relativo**.

La Arquitectura Transformer

Vaswani et al. (2017), *Attention is all you need*: codificador–decodificador **solo con atención y capas feed-forward**, sin convolución ni recurrencia.



Bloques del Codificador y del Decodificador

Bloque del codificador

$$\mathsf{Z} = \mathrm{LN}\big(\mathsf{X} + \mathrm{Drop}(\mathrm{MHA}(\mathsf{X}, \mathsf{X}, \mathsf{X}))\big)$$
$$\mathsf{Y} = \mathrm{LN}\big(\mathsf{Z} + \mathrm{Drop}(\mathrm{FFN}(\mathsf{Z}))\big)$$

Cada subcapa entrega dimensión d para poder sumar el residual. El bloque **no cambia la forma** del tensor.

Bloque del decodificador

$$\mathsf{Z_1} = \mathrm{LN}\big(\mathsf{X} + \mathrm{Drop}(\mathrm{MHA}_{\textbf{enmasc.}}(\mathsf{X}, \mathsf{X}, \mathsf{X}))\big)$$
$$\mathsf{Z_2} = \mathrm{LN}\big(\mathsf{Z_1} + \mathrm{Drop}(\mathrm{MHA}(\mathsf{Z_1}, \mathsf{H}, \mathsf{H}))\big)$$
$$\mathsf{Z_3} = \mathrm{LN}\big(\mathsf{Z_2} + \mathrm{Drop}(\mathrm{FFN}(\mathsf{Z_2}))\big)$$

H = salida del codificador, usada como claves y valores.

Embeddings

Se multiplican por $\sqrt{d}$ antes de sumar P (cuyos valores están en $[-1, 1]$), para que la posición no domine la señal del token.

FFN Posicional, Residuales y Normalización de Capa

FFN posicional

El **mismo** MLP de dos capas se aplica a cada posición por separado:

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Entradas idénticas en posiciones distintas dan salidas idénticas; la mezcla entre posiciones la hace solo la atención.

Add & Norm

- **Residual**: $x + \text{subcapa}(x)$ facilita el flujo de gradientes en pilas profundas.
- **Layer norm**: normaliza sobre las *características* de cada ejemplo.
- Batch norm normaliza sobre el lote; en secuencias de largo variable es inestable, por eso se prefiere layer norm.

Los Tres Tipos de Atención en el Transformer

Capa	Consultas	Claves / valores	Máscara
Auto-atención del codificador	codificador	codificador	padding
Auto-atención del decodificador	decodificador	decodificador	causal + padding
Atención codificador–decodificador	decodificador	salida del codificador H	padding de la fuente

Configuración del ejemplo EN→FR de d2l

2 bloques de codificador y 2 de decodificador, 4 cabezas, dimensión 256, dropout 0.2, lr $= 0.001$, 30 épocas, recorte de gradiente 1. Los mapas de atención del codificador–decodificador muestran el alineamiento objetivo $\rightarrow$ fuente.

En la práctica

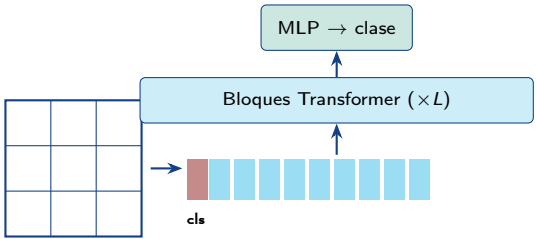
Muchos modelos usan solo el codificador (BERT) o solo el decodificador (GPT): ver la sección de preentrenamiento.

Vision Transformer (ViT)

Una imagen se trata como una **secuencia de parches**, igual que el texto es una secuencia de tokens.

1. Imagen $h \times w \times c$ dividida en parches $p \times p$: $m = hw/p^2$.
2. Cada parche aplanado (cp^2) se proyecta linealmente a $\mathbb{R}^d$ (equivale a una convolución con *kernel* y *stride* $= p$).
3. Se antepone un token `<cls>` y se suman embeddings posicionales aprendibles.
4. Pila de bloques Transformer (codificador).
5. Cabeza de clasificación sobre la salida de `<cls>`.

$$Z_0 = [x_{\text{cls}}; X_p E] + E_{\text{pos}}$$



ViT: Detalles y Lecciones

Bloque *pre-norm*

$$\mathbf{Z}' = \mathbf{Z} + \mathrm{MSA}(\mathrm{LN}(\mathbf{Z}))$$
$$\mathbf{Z}'' = \mathbf{Z}' + \mathrm{MLP}(\mathrm{LN}(\mathbf{Z}'))$$

La normalización va *antes* de cada subcapa (entrena mejor que post-norm). El MLP usa GELU.

Ejemplo d2l (Fashion-MNIST)

Imagen 96, parche 16 $\Rightarrow m = 36$ parches; 8 cabezas, 2 bloques, 512 ocultas.

Sesgo inductivo

En conjuntos pequeños o medianos, ViT **no supera** a una ResNet: le faltan la localidad y la invarianza a traslaciones de la convolución.

Con datos masivos

Con modelos y conjuntos mucho más grandes (cientos de millones de imágenes), ViT supera claramente a las ResNet.

Costo cuadrático en el número de parches. Variantes: DeiT (eficiencia de datos), Swin (ventanas desplazadas).

Tres Familias de Transformers Preentrenados

	Solo codificador	**Codificador–decodificador**	**Solo decodificador**
Ejemplo	BERT	T5	GPT, GPT-2, GPT-3
Atención	bidireccional	bidireccional + causal	causal
Preentrenamiento	lenguaje enmascarado (MLM)	corrupción de tramos	modelado de lenguaje autorregresivo
Uso típico	clasificación, etiquetado	texto a texto (traducción, resumen)	generación, *in-context learning*

Idea común

Preentrenar con texto sin etiquetas (autosupervisión) y luego **adaptar** a tareas específicas: ajuste fino o, en modelos muy grandes, solo *prompting*.

BERT, T5 y GPT: Objetivos de Preentrenamiento

BERT (MLM)

Se reemplazan tokens al azar por `<mask>` y se predicen con entropía cruzada, usando contexto a ambos lados.

"I `<mask>` this red car" $\rightarrow$ *love*

Ajuste fino: capa nueva sobre `<cls>`, entrenada junto con los parámetros preentrenados.

T5 (texto a texto)

Tramos consecutivos enmascarados comparten un token centinela; el decodificador los reconstruye.

Entrada: "I `<X>` this `<Y>`"
Salida: "`<X>` love `<Y>` red car `<Z>`"

Toda tarea es texto $\rightarrow$ texto, sin capas específicas.

GPT (autorregresivo)

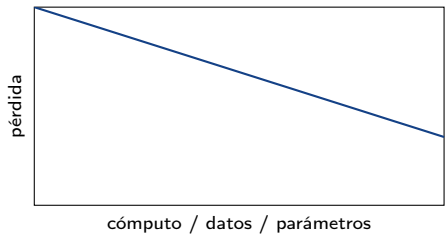
El objetivo es la entrada desplazada un token; atención causal.

$$\max_{\theta} \sum_{t} \log p_\theta(x_t \mid x_{<t})$$

GPT-2: 1.5 B parámetros; ejecuta tareas sin actualizar pesos. GPT-3: *zero-*, *one-* y *few-shot* vía contexto.

Escalamiento

- Kaplan et al. (2020): el desempeño sigue una **ley de potencias** en el tamaño del modelo, el tamaño del conjunto de datos y el cómputo.
- Los modelos más grandes son más **eficientes en muestras**.
- Capacidades nuevas aparecen con la escala: *in-context learning* sin actualizar parámetros.



Esquema cualitativo de una ley de potencias (recta en escala log–log); no son datos reales.

Laboratorio 3.3: de las Diapositivas al Código

Cuaderno `labs/3.3_AtencionPyTorch.ipynb` (basado en el cap. 3 de *Build a Large Language Model From Scratch*, S. Raschka).

Sección del lab	Qué se implementa	Diapositivas
1. Auto-atención simple	$Z = \mathrm{softmax}(XX^\top)X$ con bucles y en forma matricial	Auto-atención simple
2. Pesos entrenables	W_q, W_k, W_v; `SelfAttention_v1/v2`; experimento de $\sqrt{d_k}$	Producto punto escalado; auto-atención con pesos
3. Atención causal	máscara $-\infty$, dropout, `CausalAttention`	Máscara antes del softmax; máscara triangular; dropout
4. Multi-cabeza	`ModuleList` vs. división de pesos con `view`	Ingenua vs. eficiente; código

Actividad del Laboratorio y Dónde Repasarla

1. **Atención escalada a mano**: implementar $\mathrm{softmax}(\mathrm{QK}^\top/\sqrt{d_k})\mathrm{V}$ con máscara opcional y compararla con `F.scaled_dot_product_attention`. *(Producto punto escalado, máscara causal.)*
2. **Comparar con** `nn.MultiheadAttention`: copiar sus pesos a nuestra clase y verificar que coinciden. *(Multi-cabeza: W_o y `head_dim`.)*
3. **Equivarianza a permutaciones** y **codificación posicional sinusoidal**: mostrar que sin posición el orden no importa y que con ella sí. *(La auto-atención no conoce el orden; codificación posicional.)*
4. **Mini bloque Transformer**: MHA + FFN con residuales y LayerNorm. *(Bloques del codificador y del decodificador; FFN, Add & Norm.)*

Convención de máscaras

En el lab la máscara booleana vale `True` donde **no** se debe atender, y se aplica con `masked_fill(..., -torch.inf)`. En la diapositiva de *masked softmax* (d2l) se usan longitudes válidas y un valor -10^6; es la misma idea.

Resumen

Atención

- Promedio ponderado de **valores**, con pesos que salen de la similitud **consulta–clave** (softmax).
- Producto punto escalado: $\mathrm{softmax}(\mathrm{QK}^\top/\sqrt{d})\mathrm{V}$.
- Atención aditiva para dimensiones distintas.
- Auto-atención: $\mathrm{Q}, \mathrm{K}, \mathrm{V}$ salen de la misma secuencia; máscara causal para modelos autorregresivos.
- Multi-cabeza: varios subespacios en paralelo.

Transformer

- Auto-atención: paralelismo y camino $\mathcal{O}(1)$, a costo $\mathcal{O}(n^2 d)$.
- Codificación posicional para recuperar el orden.
- Residuales + layer norm + FFN posicional.
- Decodificador con máscara causal.
- Base de BERT, T5, GPT y ViT.

Ejercicios Teóricos Sugeridos

1. Demuestre que con $a(\mathbf{q}, \mathbf{k}_i) = \mathbf{q}^\top \mathbf{k}_i$ y $\mathbf{k}_i = \mathbf{v}_i$ se cumple $\nabla_{\mathbf{q}} \mathrm{Attention}(\mathbf{q}, \mathcal{D}) = \mathrm{Cov}_{p(\mathbf{k}_i;\mathbf{q})}[\mathbf{k}_i]$.
2. Muestre que $\mathrm{Var}(\mathbf{q}^\top \mathbf{k}) = d$ si las componentes son independientes con media 0 y varianza 1.
3. Reemplace la atención aditiva por producto punto escalado en el modelo de Bahdanau. ¿Cambia la eficiencia del entrenamiento?
4. Visualice los pesos por cabeza en un Transformer entrenado. ¿Qué cabezas podrían podarse?
5. Proponga una codificación posicional aprendible y compárela con la sinusoidal.
6. En ViT, sustituya el token `<cls>` por *mean pooling*. ¿Qué cambia?

Referencia: Zhang et al., *Dive into Deep Learning*, cap. 11, `d21.ai`. Vaswani et al. (2017); Bahdanau et al. (2014); Dosovitskiy et al. (2020); Devlin et al. (2018).